

Citizen Developers and Vibe Coders

Purpose

AI coding assistants have made it possible for many more people to write useful software. This is good for the business: employees can automate repetitive work, prototype ideas quickly, and solve local problems without waiting for a formal development team to become available.

That same capability also creates application security risk when software is created outside of a documented software development lifecycle, outside of source control, or without review of authentication, authorization, data access, secrets handling, and deployment practices.

The goal is not to stop citizen developers from building useful things. The goal is to bring that work into the open, help citizen developers use safe engineering practices, and apply lightweight security controls that scale with the risk of the application.

Definitions

Citizen developer: A person who is not formally part of a software engineering team but creates code, scripts, automations, applications, APIs, reports, or tools to support business work.

Vibe coding: Code created with heavy assistance from AI tools, often through iterative prompting and testing instead of traditional software design and development practices.

Shadow IT: Software, infrastructure, automations, integrations, or data workflows created or operated outside of approved governance, visibility, ownership, or security processes.

Why This Matters

AI can help almost anyone write code. That makes software development more accessible, but it also means business-critical or data-sensitive tools may be created by people who have not been taught secure development practices.

Common risks include:

- Code is not stored in Git or any other version control system.
- No one knows who owns, maintains, or approves the application.
- Authentication is missing, homegrown, or inconsistently applied.
- Authorization checks are missing or do not enforce least privilege.
- The application grants access based on assumptions instead of authoritative identity or group membership.
- Sensitive data is exposed through a script, dashboard, API, or web application.
- Secrets are hardcoded in source code, configuration files, scripts, or local machines.
- Dependencies, infrastructure definitions, and deployment settings are not scanned.
- AI coding agents introduce libraries, packages, or transitive dependencies whose origin, maintainer history, or security posture is not well understood.
- Code is left only on a local laptop, in an AI tool workspace, or in another location that is not protected and monitored like enterprise source code.

- The application contains business logic flaws that automated scanners cannot understand.

For example, a vibe-coded application may appear to check whether a user should access loan data, but fail to validate the user's actual role in Active Directory. Static and dynamic scanners may find framework vulnerabilities, exposed secrets, or common injection flaws, but they usually will not understand whether the application enforced the correct business authorization rule.

Security Position

Total bans on vibe coding are likely difficult to enforce and may push useful development further into shadow IT. A better approach is to establish trust, make safe development easier, and give citizen developers a clear path to bring their work into approved security visibility.

Application Security should connect with citizen developers, teach practical software development practices, and help them adopt controls that make sense for the risk of what they are building.

As the citizen developer and vibe-coding ecosystem grows, these expectations should be shared broadly with the Lakeview community so teams understand the approved path before their tools become critical business workflows.

This approach:

- Allows useful business automation and application development to continue.
- Helps teams learn Git and preserve source code history.
- Ensures code is persisted in enterprise-recognized repositories that are protected, monitored, and subject to scanning.
- Enables Wiz SAST, SCA, IaC, and secrets scanning when code is stored in approved repositories.
- Creates an opportunity to review authentication, authorization, data access, architecture, and deployment assumptions.
- Reduces shadow IT by building a cooperative relationship instead of relying only on enforcement.

What Counts as an Application?

Citizen-developed software exists on a spectrum. Security requirements should scale with business impact, data sensitivity, and exposure, but small tools should not be ignored only because they do not look like traditional applications.

Examples that may require review include:

- A PowerShell script that automates a business process.
- A PowerShell script that hosts a lightweight API.
- A Python Flask server with two endpoints.
- A Python Flask server with a frontend.
- A local desktop tool used by multiple employees.
- A report generator or data transformation script that handles sensitive data.
- A low-code or AI-generated workflow that connects to internal systems.

If code is used by others, handles business data, exposes an interface, authenticates users, authorizes access, runs on a schedule, or affects a business process, it should be visible to the appropriate technology and security teams.

Required Baseline Practices

Citizen-developed projects should follow these baseline practices unless an exception is documented and approved.

1. Identify an owner

Each project must have a named business owner and technical owner. Ownership should include who maintains the code, who approves changes, and who can answer questions about business purpose and data access.

2. Use approved source control

Source code should be stored in an approved Git repository. Git history provides accountability, recovery, change tracking, and a path to automated scanning. Code created with AI agents should not live only in a chat transcript, local folder, AI workspace, personal cloud storage, or unmanaged repository.

3. Enable automated scanning

Repositories should be onboarded to available security scanning, including Wiz SAST, SCA, IaC, and secrets scanning where supported. Findings should be reviewed and remediated according to standard vulnerability management expectations. Using enterprise-recognized repositories helps ensure that citizen-developed code receives the same scanning and visibility as other software.

4. Use approved authentication

Applications that identify users should use approved enterprise authentication patterns. Developers should avoid custom login systems, shared passwords, local-only user lists, or unauthenticated access to internal tools.

5. Enforce authorization explicitly

Access to sensitive functions or data must be enforced through clear authorization checks. Role or group checks should use authoritative sources such as Active Directory or approved identity services. The application should not assume access based only on network location, obscurity, user input, or frontend controls.

6. Protect secrets

Secrets must not be committed to source code. API keys, passwords, tokens, certificates, and connection strings should be stored in approved secret management or configuration systems.

7. Review dependencies introduced by AI agents

Citizen developers should understand what libraries and packages their AI tools add to a project. Package names, package managers, versions, maintainers, licenses, and transitive dependencies should be visible through source control and dependency scanning. Developers should avoid accepting AI-suggested packages without understanding why they are needed and whether approved alternatives exist.

8. Understand the data

Projects must identify what data they access, process, store, transmit, or display. Sensitive, regulated, customer, loan, employee, financial, or confidential business data requires stronger review.

9. **Document runtime and deployment**

The team should document where the application runs, how it is deployed, who has administrative access, what systems it connects to, and how it can be disabled if needed.

10. **Review high-risk logic**

Security review should include business logic that scanners cannot reliably detect, especially authentication decisions, authorization decisions, data filtering, workflow approvals, and access to sensitive resources.

11. **Prefer isolated development environments for higher-risk work**

Projects that use sensitive data, privileged credentials, internal APIs, or production-adjacent systems should use a sandboxed development environment where practical, such as GitHub Codespaces or a stripped-down virtual machine. This lowers the blast radius if a malicious dependency, compromised package, or hostile build script runs during development.

12. **Containerize applications where appropriate**

Containerizing applications can help standardize runtime dependencies and reduce the number of unnecessary tools available to an attacker. Teams should prefer minimal base images and include only the packages needed for the application to run.

13. **Scale controls with risk**

A one-person script that does not handle sensitive data may only need basic source control and secrets hygiene. A web application used by a department to access sensitive data may require architecture review, formal testing, logging, monitoring, and operational support.

Why Tools Are Necessary but Not Sufficient

Wiz SAST, SCA, IaC, and secrets scanning provide important coverage once code is stored in a repository. DAST can add additional runtime testing and may find vulnerabilities that static analysis misses.

However, scanning alone cannot solve this problem.

SAST tools generally inspect code patterns and dependencies. DAST tools generally test a running application from the outside. Both are valuable, but neither can reliably determine whether the application implemented the correct business rule, enforced the right Active Directory role, or prevented a user from accessing loan data they should not see.

Because of this, citizen-developed projects need both:

- Automated scanning for known vulnerability classes.
- Human review of architecture, authentication, authorization, data access, and business logic.

Supply Chain Risk and Blast Radius

Citizen-developed and AI-assisted projects should assume that supply chain attacks are possible. It is not realistic to predict every npm, PyPI, Maven, container, or operating system package that may be compromised next. It is realistic to reduce the impact if a compromise succeeds.

Campaigns such as Shai-Hulud demonstrate why this matters: compromised packages and developer tooling can steal credentials, abuse code repositories, poison CI/CD workflows, and propagate through package ecosystems. Publicly described malware and copied attack techniques also make it easier for additional attackers to stage similar campaigns.

AI agents can make this risk easier to miss because they may add dependencies quickly and confidently. A developer may not know whether a suggested package is popular, maintained, recently created, typosquatted, compromised, or pulling in risky child dependencies. This is another reason code and dependency manifests should be committed to approved repositories where scanning, review, and history are available.

The goal is to reduce blast radius. If a compromised package executes on a laptop containing many API keys, cloud credentials, personal files, and administrative tools, response may require broad credential rotation and endpoint investigation. If that same package executes in a sandboxed development environment with only the minimum required access, response can be narrower and faster.

Proactive Defenses

Preferred proactive controls include:

- Use sandboxed development environments such as GitHub Codespaces or stripped-down virtual machines for projects with meaningful data, credential, or production access.
- Provide a secure development "happy path" so developers have the tools they need in the sandboxed environment. If the approved path is too hard to use, teams will work around it.
- For higher-risk development, consider restricting or prohibiting local laptop development when a usable sandboxed alternative exists.
- Use enterprise package management, repository mirroring, or artifact management platforms such as Nexus or JFrog Artifactory to improve visibility and control over dependency versions.
- Consider cleanroom or hardened package providers such as Seal Security or Chainguard where appropriate. These services can help reduce dependency compromise risk and may provide backported CVE fixes without requiring immediate application code changes.
- Use dependency cooldowns, such as the model described at <https://cooldowns.dev/>, to avoid automatically pulling brand-new package versions before the ecosystem has had time to detect malicious releases.
- Prefer stable, non-bleeding-edge dependency versions unless a project has a clear reason to adopt a newer version.
- Containerize applications where appropriate, and use minimal base images to reduce unnecessary binaries, tools, and packages.
- Keep dependency manifests and lockfiles in source control so changes to packages are visible, reviewable, and scannable.

These controls do not guarantee prevention. They make successful attacks less damaging and easier to investigate.

Reactive Defenses

Reactive controls are still necessary because some attacks will only be understood after they happen.

Recommended reactive steps include:

- Use the SOC team and Wiz Code to determine whether enterprise repositories use a compromised dependency, version, container image, or infrastructure pattern.
- Use Wiz Mika or equivalent investigation capabilities to ask whether the organization is affected by a specific campaign, package, version, or indicator.
- Search for indicators of compromise, including suspicious hostnames, IP addresses, files written to disk, unexpected build scripts, unusual package install behavior, or unexpected outbound network connections.
- Rotate credentials quickly when there is evidence that a development environment, build environment, package install process, or application runtime may have exposed secrets.
- Review source control history, dependency manifests, package lockfiles, CI/CD logs, and endpoint telemetry to understand when a compromised dependency was introduced and where it ran.

Reactive detection is important, but it should not be the primary strategy. Proactive isolation and dependency governance reduce the amount of damage that reactive response must contain.

Engagement Model

When AppSec, IT, or another technology team discovers a citizen-developed project, the preferred response is to engage constructively.

Recommended steps:

1. Meet with the project owner and understand the business purpose.
2. Identify users, data sources, sensitive data, integrations, and deployment location.
3. Help the owner move code into an approved Git repository if it is not already there.
4. Enable available Wiz SAST, SCA, IaC, and secrets scanning.
5. Review authentication and authorization design.
6. Review secrets handling, dependency management, and deployment practices.
7. Review dependencies introduced by AI agents, including package managers, lockfiles, and transitive dependencies.
8. Determine whether the project should use a sandboxed development environment, enterprise package repository, dependency cooldown, containerization, or other supply chain control.
9. Document any required remediation.
10. Agree on a support and ownership model.
11. Set expectations for future changes and recurring review when risk warrants it.

The tone of the engagement matters. Citizen developers are often solving real business problems. If security teams treat them as partners, they are more likely to disclose projects early, ask for help, and adopt secure practices.

Escalation Criteria

Additional review or escalation may be needed when a citizen-developed project:

- Handles sensitive, regulated, customer, employee, financial, or loan data.
- Provides access to internal systems or production data.

- Exposes a web application, API, or network listener.
- Is used by multiple employees or supports a recurring business process.
- Uses privileged service accounts or shared credentials.
- Runs package install, build, or deployment steps on developer laptops with access to sensitive credentials.
- Depends on newly published, unreviewed, unusual, or AI-selected third-party packages.
- Changes business records, approvals, payments, access decisions, or operational workflows.
- Cannot be moved into source control or scanned.
- Has no clear owner or support model.

Escalation should focus on reducing risk and clarifying ownership, not punishing the act of development.

Practical Intake Checklist

Use this checklist when onboarding a citizen-developed project:

- Project name:
- Business owner:
- Technical owner:
- Business purpose:
- Users or groups who need access:
- Data accessed, processed, stored, or displayed:
- Sensitive data classification:
- Source code repository:
- Scanning enabled:
- Runtime or hosting location:
- Authentication method:
- Authorization model:
- Secrets storage method:
- External dependencies:
- Package managers and lockfiles:
- AI-suggested packages reviewed:
- Enterprise package repository or mirror used:
- Sandboxed development environment used:
- Container image or base runtime:
- Internal systems or APIs used:
- Deployment process:
- Logging or monitoring:
- Known risks or open remediation items:
- Next review date:

Example: Desktop Engineering CommandCenter

Application Security partnered with the Desktop Engineering team to help them start using Git and begin scanning their "commandcenter" project with Wiz. This is the preferred model: connect with the team, teach the practices, bring the code into security visibility, and preserve the business value of the project.

Thank you to the Desktop Engineering team for being open to learning and partnering on a safer way to build.

Summary

Citizen developers and vibe coders are not the problem. Hidden, unmanaged, and unaudited software is the problem.

The best way to prevent shadow IT is to build trust with the people creating useful tools, teach them how to develop safely, and make the secure path practical. Version control, automated scanning, authentication review, authorization review, and architecture conversations give the business the benefits of AI-assisted development without pretending that tools alone can understand every security decision.